```
  return $office->cancelled;
});
```

The Eloquent collection class has a few extra methods in addition to those offered by Laravel's standard collection class that are specifically designed for working with collections of Eloquent models.

You may loop over collections as if they were an array because Laravel's collections all adhere to PHP's iterable interfaces:

```
foreach ($offices as $office) {

  echo $office->id;
}
```

Chunking Results

If you try to use the **`all`** or **`get`** methods to load tens of thousands of Eloquent entries, your application can run out of memory. The **`chunk`** method can be used to process a lot of models more quickly than these other techniques.

A closure will receive a subset of Eloquent models that were retrieved by the **`chunk`** method and processed. When working with a large number of models, the **`chunk`** approach will result in much less memory use because just the most recent chunk of Eloquent models is obtained at a time:

```
use App\Models\Office;
use Illuminate\Database\Eloquent\Collection;
Office::chunk(200, function (Collection $offices) {
  foreach ($offices as $office) {
    // Code Logic
  }
});
```

The number of records you desire to receive each **chunk** is the first input supplied to the **`chunk`** method. For each chunk that is retrieved from the database, the closure supplied as the second parameter will be called. Each chunk of records supplied to the closure will be retrieved using a database query.

Use the **`chunkById`** method if you want to filter the outcomes of the **`chunk`** method based on a column that will also be updated when you iterate through the outcomes. In some situations, applying the **`chunk`** technique